Consider an operating system managing multiple files and devices for concurrent users.

(a) Categorize system calls used for file and device management.

File creation & Deletion:-

Create or remove files

ex: create(), unlink()

File Access :-

open and close files

ex: open(), close()

files Read and write :-

Read / write file contents

ex: read(), write()

file positioning :-

Move file pointer

ex :- lseek()

file Attributes :-

Get or modify permission

ex :- Stat(), Chmod()

Device Management :-

Access hardware devices

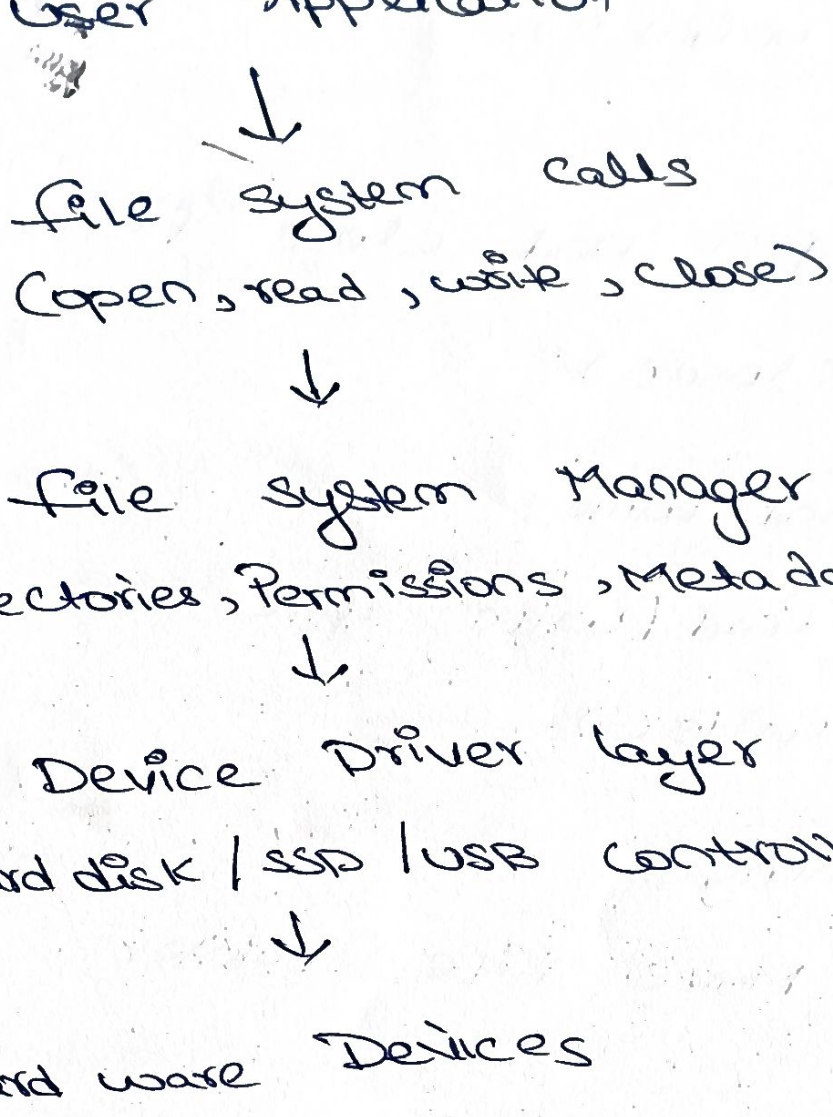
ex :- ioctl(),

Buffer Management :-

Transfer data between device and memory.

Ex :- Device drivers

(b) Illustrate how the OS handles the files operations using a layered structure diagram



Working :-

→ User requests a file operation
→ System call enters the kernel.
→ File system checks permissions and file location.
→ Data is returned to the application

features :-

→ Deterministic response time.
→ Executes tasks within fixed deadlines.
→ Supports preemptive, priority-based scheduling
→ low interrupt latency.
→ High reliability and stability
→ Efficient multitasking
→ Suitable for mission-critical and Embedded Systems

Applications :-

→ Industrial automation
→ Robotics
→ Medical equipment
→ Traffic signal systems

(b) Draw a diagram representing the interaction between hardware, kernel and user processes

User processes
↓
System Calls
↓
RTOS kernel
→ Task scheduler
→ Device Drivers
↓
Hard ware

(c) Write a pseudo-code or C-based System call sequence to open, read and close a file

```
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    char buffer[100];
    fd = open("sample.txt", O_RDONLY);
file
    read(fd, buffer, sizeof(buffer));
file
    close(fd);
file
    return 0;
}
```

open() → read() → close()

3) Assume a real-time operating system used in an embedded system controlling industrial machinery.

(a) Identify the type of OS and explain why it is suitable for real-time application

Type: Real-Time operating System

(c) List and explain system calls required for Process synchronization on this system

fork() → Creates a new process

wait() → waits for a child process to Complete

Sem_init() → Initializes a semaphore

Sem_post() → unlocks the semaphore after Completing the task.

Pthread_mutex_lock() → locks a mutex to prevent Concurrent access

Pthread_mutex_unlock() → Releases the mutex